

NOT and NEG

One's complement $\rightarrow$ two's complement
 $\hookrightarrow$ mov AL, 0001011b
 not AL ; AL = 1100100b
 ret
 neg AL ; AL = -5

PROGRAM CONTROL INSTRUCTION

Controlling the program flow is very important, where your program can make decisions based on particular condition. In the Assembly language, flow control instruction $\rightarrow$ Jump

Unconditional Jump

JMP (label), x to declare a label (name:). A label can be declared on a separate line or before any other construction.
 ex: label1:
 a:

Conditional Jump

Conditional jumps are always short jumps in 8086-8088. These instructions are divided in three groups:

- Just tests single flag
- compares numbers as signed
- " as unsigned.

SYNTAX

<conditional jump instruction> <label>
 $\hookrightarrow$ if it's true

$\hookrightarrow$ the label occurs

$\hookrightarrow$ if it's false, the next sequential step in the program executes.

Instruction Description

JZ, JE Jump if zero/equal
 JC, JB, JNAE Jump if carry/below/not above equal
 JS Jump if sign
 JO Jump if overflow
 JPE, JP Jump if parity even

Condition

$Z=1$
 $C=1$
 $S=1$
 $O=1$
 $P=1$

Opposite

$0 \leftarrow JNZ, JNE$
 $0 \leftarrow JNC, JNB, JAE$
 $0 \leftarrow JNS$
 $0 \leftarrow JNO$
 $0 \leftarrow JPO, JNP$

JG, JNLE Jump if greater/not less or equal
 JL, JNGE Jump if less/not greater or equal

JA, JNBE Jump if above/not below or equal
 JB, JNAE, JC Jump if below/not above or equal/carry

* Signed numbers compared (greater than, less than) etc.
 * Unsigned numbers compared (above, below)

require $\rightarrow$ gender problems
Cumbersome $\rightarrow$ kullanma

~ MICROCONTROLLER PROGRAMMING ~

The Mechanical Age

- Abacus $\rightarrow$ invented by Babylonians, first mechanical calculator
- Pascal invented a calculator. Same principle used in automobile odometer) basis of all mechanical calculator. (PASCAL programming language)
- Analytical Engine (Charles Babbage and Ada Byron)
 - * programmable calculating machine
 - steam-powered mechanical computer
 - navigational tables
 - punched cards
 - stored 1000 20-digit decimal numbers
- Colossus - Alan Turing (1943)
 - broke military codes, by ENIGMA
 - vacuum tubes
 - not programmable
 - could not solve other problems (special-purpose computer)
- ENIAC, Univ. of Pennsylvania
 - over 30 tons
 - 100,000 operation per second
 - programmed by rewiring its circuits $\rightarrow$ transistor were developed.
- 1971, the first microprocessor by Intel (4004)
- Von Neumann machines (computer), first modern person to develop a system to accept instructions and store them in memory
- Assembly language was then used to simplify entering binary code.
 - still plays important role $\rightarrow$ ex. video games

Reminder

- A bit (0 or 1) -
- 4 bit wide (nibble) - - -
- 8 bit wide (byte) - - - - -
- * A word $\rightarrow$ 2 bytes (16 bits)

The Microprocessor Age

- 1971, Intel 4004 and 4040 - 1972, Intel 8008
 - 4 bits microprocessor
 - 4096 (4 bit) memory location
 - 45 instruction
 - still used 4 bit $\rightarrow$ microwave
- 1978, Intel 8086
 - 16 bit m.p.
 - addressed 1M memory location
 - over 20000 inst. variations
- Such microprocessors are called CISC (complex instruction set computers)
 - alternative is RISC (reduced instruction set computers)

bit (b)
byte (B)
1KB = 2^{10} Bytes
1MB = 2^{20} Bytes
1GB = 2^{30} Bytes

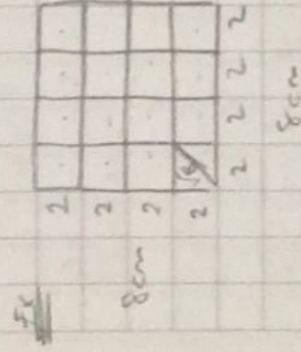
Ex 366 people

≥ 2 birthday same

$$\left\lceil \frac{366}{365} \right\rceil = 2 //$$

Ex $S = \{1, 2, 3, \dots, 20\}$, if we pick 11 numbers, we are guaranteed that the sum of two picked numbers is 21.

$$\{1, 20\}, \{2, 19\}, \{3, 18\}, \{4, 17\}, \{5, 16\}, \{6, 15\}, \{7, 14\}, \{8, 13\}, \{9, 12\}, \{10, 11\}, \left\lceil \frac{11}{2} \right\rceil = 2 //$$



$\Rightarrow 2 \text{ dots} < \sqrt{8} \text{ cm}$

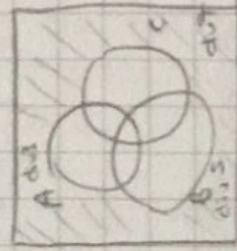
min. num of dots: 2

$$\left\lceil \frac{17}{16} \right\rceil = 2 //$$

$$\text{So } n = 17 //$$

Principle of Inclusion and Exclusion

$U = \text{between } 1 \text{ and } 1000$



Conditions

C_1 : div by 3

C_2 : div by 5

C_3 : div by 7

$$N = 1000 - |A| - |B| - |C| + |A \cap B| + |A \cap C| + |B \cap C| - |A \cap B \cap C|$$

$$\bar{N} = S_0 - S_1 + S_2 - S_3 + \dots - (-1)^{n-1} S_n$$

Inclusion-Exclusion Formula

$\downarrow$ number of condition

$N - \bar{N} = \text{at least one condition}$

Ex $x_1 + x_2 + x_3 + x_4 = 18 \quad \left(\frac{21}{3} \right) \quad \forall x_i \geq 0 \quad 1 \leq x_i \leq 4$

Ex $x_1 + x_2 + x_3 + x_4 = 18$

universal set: $(\mathbb{N})$

$C_1: x_1 \geq 8$
 $C_2: x_2 \geq 8$
 $C_3: x_3 \geq 8$

Microprocessor

- a programmable digital electronic component

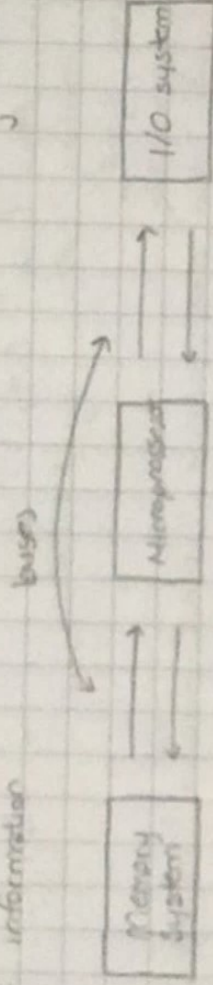
- The only difference between microcontroller and microprocessor

↓
three parts from microprocessor
and additional part
(ROM, RAM, etc)

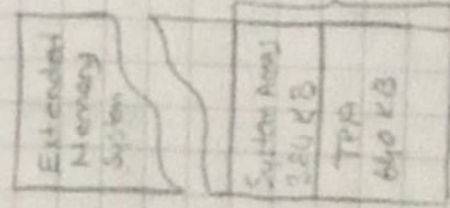
↓
ALU
control unit
Register

The Microprocessor-Based Personal Computer System

- filled large areas to small desktop computer because of microprocessors
- a bus is the set of common connections that carry the same type of information



The memory map of PC



The TPA (Transient program area)

- holds DOS (disc operating system)
- not applicable in Windows
- store DOS application program

The System Area

- smaller than TPA, just as important
- contains programs or read-only (ROM) or flash memory and read/write (RAM) memory for data store
- Start from A0000H to CFFFFH

Windows Systems

- has two main areas, TPA and System Area
- the difference between them and DOS memory map are SIZES and LOCATIONS of these areas
- I/O space

- allow the microprocessor to communicate with the outside world
- from 0000H to FFFFH
 - I/O port address is similar to memory addresses
 - instead of memory, it addresses an I/O device

via - → - architecture
request → internet
errors → birthday
vary → degree
take clock.

The Microprocessor

- Called the CPU (central processing unit)
- controlling element in a computer system
- controls memory and I/O with buses
- memory and I/O controlled via instructions stored in memory, executed by the microprocessor

MIP: three main tasks

1. data transfer
2. simple arithmetic and logic operations.
3. program flow via simple decisions.

SOFTWARE (PROGRAMS - group of instructions)

Buses

- Transfer address, data, and control info. between microproc., memory, I/O.
- Three buses exist for transfer of information

- The ADDRESS BUS request a memory location from the memory or an I/O location from I/O device

- If I/O is addressed, the address bus contains a 16-bits I/O address (0000H - FFFFH)

- If memory is addressed, the bus contains a memory address of 20-bits for 0086)

- The DATA BUS transfers information between the microprocessor and its memory and I/O address space.

- data transfers vary in size from 8-bits to 64-bits. (16-bits - 0086)
- CONTROL BUS lines select and cause memory or I/O to perform a read or write operation

- In most computer systems, there are four control bus connections:

MRDC (memory read control)

MWDC (memory write control)

IORC (I/O read control)

IOWC (I/O write control)

- * The microprocessor reads a memory location by sending the memory an address through the address bus.

Next, it sends a memory read control signal to cause the memory to read data

Data read from memory are passed to the microprocessor through the data bus.

Whenever a memory write, I/O write or I/O read occurs, the same sequence ensues

The programming model

- 8086 through Core2 considered program visible.
- particular registers are used during programming and are specified by the instructions.
- Other registers considered to be program invisible
- not addressable directly during applications programming.

Multipurpose Registers

1. RAX (64-bits)

for 8086 $\rightarrow$ 16-bits AX

EAX (32-bits) (accumulator)

8-bits registers AH and AL

The accumulator is used for instruction such as multiplication, division, and some of the adjustment instruction.

2. RBX, addressable as RBX, EBX, BX, 3+, BL

- BX register (base index) sometimes holds offset address of a location in the memory system

3. RCX, as RCX, ECX, CX, CH, CL

- a counter general-purpose register that also holds the count for various instructions.

4. RDX, as RDX, EDX, DX, DH, DL

- a (data) general-purpose register

- holds a part of the result from a multiplication or part of dividend before a division.

5. BP - points to a memory (base points) location for memory data transfers

6. DI - often addresses (destination index) string destination data for the string instructions

7. SI - the (source index) register addresses source string data for the string instructions

C (Carry) holds carry after addition or borrow after subtraction also indicates error conditions.

P (Parity)

odd parity $\rightarrow$ 1110 1010 $\Rightarrow$ P=0

even parity $\rightarrow$ 0000 0101 $\Rightarrow$ P=1

A (Auxiliary carry) holds the carry (bit position 3 and 4 of the result)

Z (Zero) shows that the result is zero

S (Sign)

holds the arithmetic sign of the result executes

T (Trap)

enables trapping through an on-chip debugging feature

I (Interrupts)

input pin

O (Direction)

select increment or decrement mode for the DI

and/or SI registers.

O (Overflow)

occurs when signed numbers are added or subtracted

- an overflow indicates the result has exceeded the capacity of the machine

Byte - Size Data

- Stored as unsigned and signed integers
- In signed integer format, the leftmost bit represent the sign bit of the number
- unsigned integers $[0, 255]$ $(2^8 - 1)$
- signed integers $[-127, 127]$ and $[0, 127]$
- Negative signed numbers $\rightarrow$ stored in two's complement form.
 - $5_{10} \rightarrow (0011)_2$ $(-3)_{10} \rightarrow (1101)_2$

Word - Size Data

- A word (16-bits) is formed with two bytes of data.
- ① The least significant byte always stored in the lowest - numbered memory location.
- ② Most significant byte is stored in the highest.
 - This method of storing a number is called the little-endian format.
- ③ Most significant data $\rightarrow$ lowest location
 - Alternate method is called big-endian format.
 - Not used with intel microprocessors.
 - Used with Motorola family of microprocessors.

Doubleword - Sized Data

- 4 bytes of memory (it is 32-bit number)
- Appears as a product after multiplication
- Appears as a dividend before division
- Define doubleword, or dd (assembler). (also use dword)

Real Numbers

- Area or floating point number contains two parts:
 - a mantissa, significant, fraction
 - exponent

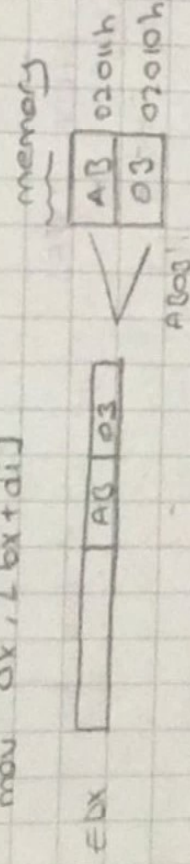
- 4 byte number $\rightarrow$ single precision
- 8 byte form $\rightarrow$ double precision

~ The Microprocessor and it's Architecture ~

Programming Structure

- The lowest - level programming lang $\rightarrow$ Machine language (only one understood by computers), impossible for humans because of 0's and 1's.
- Assembly: same as machine language, but instructions and variables have names instead numbers.
- Every CPU has own unique machine language. (rewritten or recompiled to run on different types of computers).
- High-level languages are independent. They can run on different types of computers.

4. mov [bx], cl
5. mov dx, [bx+di]



Data Structure

Used to specify how information is stored in a memory array.

- a template for data.

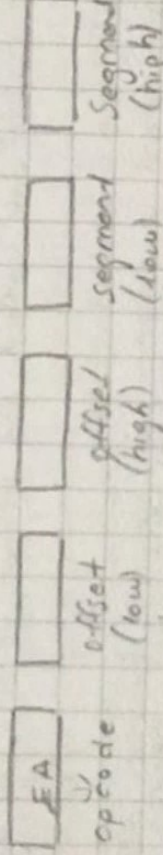
The start of a structure is identified with the `STRUCT` assembly language directive and the end with the `ENDS` statement.

2. Program Memory - Addressing Modes

- Direct Program Memory Addressing

The microprocessor uses this for but not as often as relative and indirect.

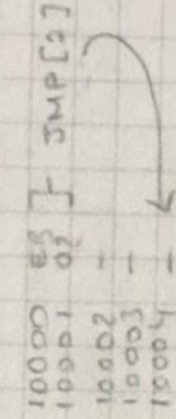
The instructions for direct p.m.a store the address with the opcode.



JMP → can jump to any memory location for the next instruction. The only other inst. using direct p.m.a is the `int` instruction or far call instruction.

- Relative P.M.A

Not available in all early microprocessors, but is available to this family of microprocessors.



JMP [di] inst.

This instruction skips over the 2 bytes of memory that follow the jmp instruction.

- Indirect P.M.A.

If a relative register holds the address, the jump is considered to be an indirect jump.

JMP [BX] → refers to the memory location within the data segment at the offset address contained in BX.

3. Stack Memory - Addressing Modes

The stack plays an important role in all microprocessors. It holds data temporarily & return address.

- LIFO (Last-in, first-out)

PUSH instruction → data are placed on the stack
POP instruction → remove data

Stack memory is maintained by two registers:

SP (stack pointer) and SS (stack segment register)

- a word of data pushed onto stack

1. the high-order 8 bits are placed in the location addressed by SP-1

2. the low-order 8 bits → SP-2

3. the SP is decremented by 2 so the next word is stored in the next available stack location.

- a data are popped from the stack.

1. the low-order 8 bits are remove

2. the high-order 8 bits are remove

3. and, SP register increment by 2

NOTE that PUSH and POP store or retrieve word of data (never bytes) in 8086.

~ DATA MOVEMENT INSTRUCTIONS

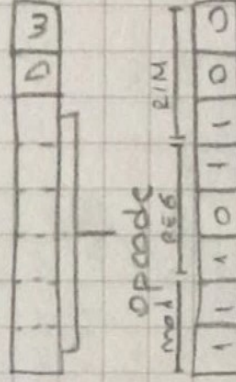
The Opcode

Select the operation performed by the microprocessor.

first 6-bits of the first byte are the binary opcode.

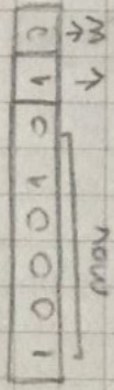
remaining 2 bits indicate the direction (D) of the data flow,

and indicate whether the data are a byte or a word (W)

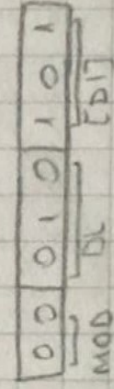


If mod = 11, register-addressing mode is selected

EX: mov DL, [DI]



D (to reg from R/M)



no displacement

Operands of MOV

REG: AX, BX, ..., AH, BH, ..., AL, BL, DL, SI, BP, SP

Memory: [BX], [BX+SI+I], variable, etc.

Immediate: 5, -24, 3Fh, 10001010b etc.

privilege $\rightarrow$ ayrılık

1. Real Mode Memory Addressing

Real mode operation allows addressing of only the first 1MB of memory space. The 1 MB is called (real memory, conventional memory, DOS memory) segment address. It defines the beginning address of any 64KB memory segment. offset address selects any location within the 64KB memory segment.

Relocation

Segment plus offset addressing allows DOS programs to be relocated in memory.

A relocated program is one that can be placed into any area of memory and executed without change.

Relocatable data are data that can be placed in any area of memory and used without any change to the program.

2. Protected Mode Memory Addressing

Allow access to data and programs located within and above the first 1MB of memory.

+ Protected mode is where Windows operates.

Selector that selects a descriptor from descriptor table.

Segment register contains selector

The descriptor describes the memory segment's location, length, and access rights.

Indirectly, the register still selects a memory segment, but not directly as in real mode.

3. Flat Mode Memory Addressing

A flat mode memory system is one in which there is no segmentation.

- does not use a segment register to address a location in the memory.

address is 40-bits

The segment register still selects the privilege level of the software

A Real mode system is not available if the processor

operates in the 64-bit mode.

A Protection and paging are allowed in the 64-bit mode.

The CS register is still used in the protected mode operation

in the 64-bit mode.

Most programs today are operated in the IA32 compatible mode.

- Current software operates properly, but this will change in

chain $\rightarrow$ indir

Segment register operand

mov seg, memory
memory, seg
seg, reg
reg, seg

seg: DS, ES, SS, and only as second operand CS

Variables

Syntax:

- can be any [name
letter or digit
- start with letter

define byte

[name] value can be any numeric value in
[name] value any supported numbering system.
define word or "q" $\rightarrow$ for variables that are
not initialized.

x Hs possible to declare unnamed variables
Other variable will have an address but
no name)

ORG Directive

It tells assembler how to handle the source code.

Arrays

Can be seen as chain of variables.

ex a DB 48h, 65h, 6Ch, 6Ch, 6Fh, 00h
b DB 'Hello', 0

b is an exact copy the a array. when assembler sees a
string inside quotes it automatically converts it to set of
bytes.

accessing

mov AL, a[3]

$\rightarrow$ can also use any of memory index
register BX, SI, DI, BP

x₁₆
mov SI, 3
mov AL, a[SI]

declaring large array (DUP)

the syntax

number DUP (value(s))

c DB 5 DUP(0) $\Leftrightarrow$ c DB 5, 0, 0, 0, 0

x If values ≥ 255 or ≤ -128 we can use DW instead of DB

Constant (EQU)

name EQU (any expression)

ex 1 equ 5 $\rightarrow$ H is functionally identical to code:

mov ax, 1

PUSH $\rightarrow$ AX, CX, DX, BX, SP, BP, SI, DI

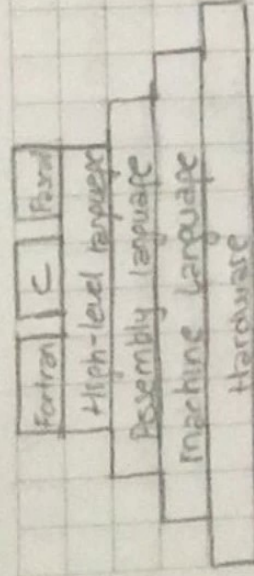
POP $\rightarrow$ DI, SI, GP, SP, BX, DX, CX, AX

cannot use declare string

mov AX, 15

fetch $\rightarrow$ gidip getirmek

* Fetching the next instruction while the current instruction executes is called Pipelining



Software Development Flow

1. The Compiler (C, C++, C#) translates source code into assembly language source code.
The Assembler translates assembly language source files into machine language (object files). Source files contain numbers corresponding to instructions.

The Linker combines object files created by the assembler into a single executable object module. It also accepts library members and output modules created by a previous linker run.

The cross-reference linker uses object files to produce a cross-reference listing showing variable and symbols, their definitions, memory locations and their references in the linker source files.

Intel - 8086

- 16-bits microprocessor
- A 20-bits external address bus gives a 1MB (segmented)
- Two separate functional units

BIU (Bus Interface Unit)

- sends out address
 - fetches instruction from memory
 - reads data from ports and memory
 - writes data to ports and memory
- $\rightarrow$ In other words, the BIU handles all transfers of data and addresses on the buses for the execution unit.

Basic Instruction Cycle

execute $\rightarrow$

fetch

decode $\rightarrow$

EU (Execution Unit)

- tells the BIU
- where to fetch instructions or data from
- decodes instruction and executes inst.
- The EU contains 16-bits ALU which can add, subtract, AND, OR, XOR, increment, decrement, complement, or shift binary numbers.

forever

1. fetch instruction
2. advance PC
3. decode
4. execute

* PC (Program Counter)

keeps the address of the instruction that is being executed

seldomly $\rightarrow$ nadiren

BCH

$$2AC = 0010\ 1010\ 1100$$
$$1000\ 0011\ 1101\ 1110 = 8D.E$$

Complements

- Two systems used to represent negative data:

$$\begin{array}{r} 1111\ 1111 \\ - 0100\ 1100 \\ \hline \end{array}$$

$$1011\ 0011$$

(radix-1 complement)

$$\begin{array}{r} 1111\ 1111 \\ - 0100\ 1000 \\ \hline \end{array}$$

$$1011\ 0111$$

(one's complement)

$$\begin{array}{r} 1011\ 1000 \\ + \\ \hline \end{array}$$

$$1011\ 1000$$

(radix-2 complement)

ASCII and Unicode Data

* American standard code for Information Interchange

- Standard ASCII code is a 7-bit code. 8. bit to hold parity
most significant

$\rightarrow$ 0 alphanumeric

Using printer parity bit $\rightarrow$ 1 graphics

* Many Windows-based applications use the UNICODE system to store alphanumeric characters

- Store each character as 16-bit data

- 0000H - 00FFH (same as ASCII)

BCD

- Stored in packed form:

• packed BCD data stored two digit per byte

• used for BCD addition and subtraction in the instruction set of microprocessor.

- Stored in unpacked form:

• stored as one digit per byte

• returned from a keypad or keyboard

- Decimal -

- Packed -

$$0001\ 0010$$

$$0000\ 0110\ 0010\ 0011$$

$$0001\ 0000$$

(two digit per byte)

- BCD data used, if there is a minimal amount of simple arithmetic to perform

- If a system requires a complex arithmetic, BCD data are seldomly used.

- Unpacked -

$$0000\ 0001\ 0000\ 0010$$

$$0000\ 0110\ 0000\ 0011$$

$$0000\ 0001\ 0000\ 0000$$

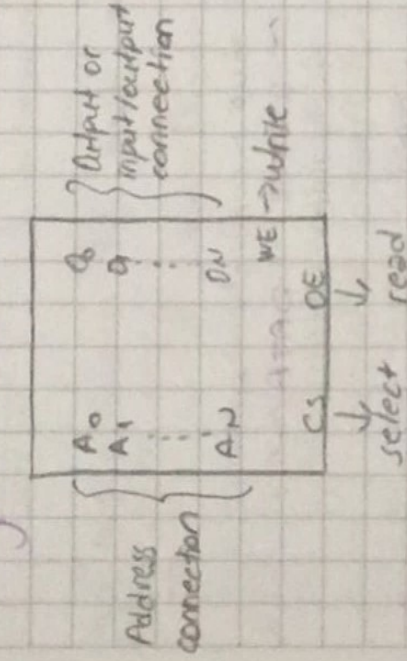
(one digit per byte)

RET

The ret instruction returns from a procedure by removing the return address from the stack and placing it into IP (near return), or IP and CS (far return)

MEMORY INTERFACE

Memory Pin Connections



- address inputs
- data outputs or input/output
- some type of selection inputs
- at least one control input to select a read or write operation

Address Connections

Memory devices have address inputs to select a memory location within the device.

A memory device with 10 address pins has its address pins labeled from A₀ to A₉

A_N → N = 10-1 (Address pin - 1)

Today memory devices have memory location starting from 1K to multiple G.

A 1K memory device has 10 address pins. (1024 memory location)

- 1024 different combinations

- if a device has 14 address pins, 2048 (2K) memory location

the number of memory location → (2^{number of pins})

Data Connections

All memory devices have a set of data outputs, or input/output

8-bits-wide memory device → D₀, D₁, ..., D₇

byte-wide memory 1K x 8

number of location → wide of location

Memory devices are often classified according to total bit capacity.

8K x 1	each location 1 bit	(8 location)	(8 bit)
1K x 8	each location 8 bits	(1 location)	(8 bit)

enable $\rightarrow$ etkinleştirme
quality $\rightarrow$ kalite

cache memory $\rightarrow$ an bellek
main memory $\rightarrow$ ana bellek

Selection Connections

Each memory device has an input that selects or enables the memory device.

- sometimes more than one.

RAM has at least one $\overline{CS}$ (chip select) or $\overline{S}$ (select)

RAM has at least one $\overline{CE}$ (chip enable)

* If more than one $\overline{CE}$ connection is present, all must be activated to read or write data.

Control Connections

All memory devices have some form of control input or inputs.

ROM $\rightarrow$ usually has one control input

RAM $\rightarrow$ usually has one or two control inputs.

~ READ DATA ~

1. Provide address
2. Enable the chip
3. Reads the data

~ WRITE DATA ~

1. Provide address
2. Provide data
3. Enable chip
4. Writes the data

ROM Memory

- permanently stores programs / data
* must not change when power disconnected. (kalıcı)

usually is used on BIOS

- just reads (read-only-memory)

- cannot be overwritten

* The ROM is programmed outside the computer and normally is only read.

RAM Memory

- both write and read

- temporarily stores programs / data

- when power disconnected, erase

SRAM (STATIC RAM)

- high quality, expensive

- data stores on transistors

so data is always protected,

when power connected.

- less power

- faster

- cache memory used

(volatile)

volatile (çocuk)

DRAM (DYNAMIC RAM)

- cheap

- data stores on capacitors

so data is mostly lost, and

must be rewritten

- more power

- slower

- main memory used

(non-volatile)

70% use → memory view
cache memory → on belief

enable → enhance
quality → rate

10	11	12	13
14	15	16	17

Address Decoding

For 1KB → 10 address

Ex

8 lane 8kb → each of them needs 13 pins. (3 lane system main system)

NAND Gate Decoder
20-bits

2K x 8 → each of 2KB device needs 11 pins. (A₀-A₁₀)
EPROM 9 address pins connected to a NAND gate decoder (A₁₁-A₁₉)

↓
do not efficient
not often used

The 3-to-8 line Decoder (74LS138)

11 is more common

Error Detection and Correction

The system is halted if an error is found in memory.

Error correction is starting to show up in new system.

SDRAM has ECC (Error Correction Code)

• correction will allow the system can continue operating.
• If two errors occur, they can be detected but not corrected.

• error correction will of course cost more in terms of extra bits

Error Corrections

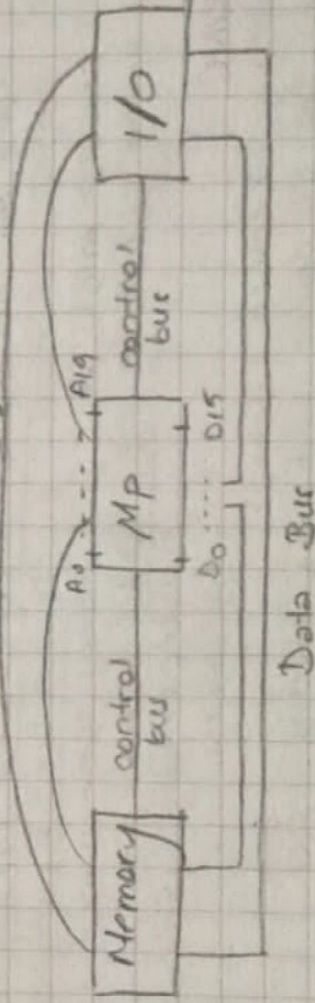
Hamming Codes

Error correction is based on Error detection and correction

Presumes one bit error mostly likely.

Detects error and position, therefore allows for correction

I/O INTERFACE



8086 → 16-bit microprocessor → 2¹⁶ fast pins - also called

signed int. $\rightarrow 4, 011116$
 unsigned int $\rightarrow -4, -2$

Multiplication and Division

The multiplication instruction contains one operand because it always multiplies the operand times the contents of register AL/AX

signed $\rightarrow$ IMUL unsigned $\rightarrow$ MUL

Algorithm: $AX = AL * \text{operand (byte)}$

MUL	REG
	MEMORY

MOV AL, 200 ; AL = 0C8h
 MOV BL, 4
 MUL BL ; AX = 0320h (800)
 RET

Division

signed $\rightarrow$ IDIV unsigned $\rightarrow$ DIV

There is no immediate division instruction available to any microprocessor.

• error types: (2)

- attempt to divide by zero
- other is a divide overflow, which occurs when a small number divides into a large number.
- The microprocessor generates an interrupt if a divide error occurs.

The Remainder

Algorithm: $AL = AX / \text{operand (byte)}$
AX = remainder (modulus)

DIV	REG
	MEMORY

Ex: MOV AX, 203 ; AX = 00CBh
 MOV BL, 4
 DIV BL ; AL = 50 (32h), AH = 3
 RET

remainder
 AH = 3

Base Logic Instructions

- AND, OR, XOR, NOT, also TEST, NEG
- Logic operations provide binary bit control in low-level software
- All logic instructions affect the flag bits
- Logic operations always clear the carry and overflow flag.

TEST

performs the AND operation function the same manner as a CMP usually followed by either JZ, JC, or JNZ instruction

TEST AL, 1 ; Test Right
 JNZ RIGHT ; if not
 TEST AL, 0 ; Test Left
 JNZ LEFT ; if not

String Data Transfers

• Before string instructions are presented, the operation of the D-flag-bit (direction), DI, SI must be understood as they apply to the string instructions.

CLD Clears D-flag $\rightarrow$ auto-increment mode

STD Sets D-flag $\rightarrow$ auto-decrement mode

DI, SI

During execution of string instruction, memory accesses occur through DI and SI registers.

ARITHMETIC and LOGIC INSTRUCTIONS

ADD $\Rightarrow$ operand1 = op1 + op2

mov AL, 5 ; al = 5

ADD AL, -3 ; al = 2

RET

ADC $\Rightarrow$ op1 = op1 + op2 + CF

stc ; set CF = 1

mov al, 5 ; al = 5

adc al, 1 ; al = 2

; al = 5 + 1 + 1

CF

ret

DEC $\Rightarrow$ op = op - 1

mov al, 255 ; al = 0FEh

dec al ; al = 0FEh

ret

Comparison (CMP)

• The comparison instruction (CMP) is a subtraction that changes only the flag bits.

- destination operand never changes.

• Useful for checking the contents of a register or memory location against another value.

• CMP is normally followed by a conditional jump instruction.

- op1 - op2

- result is not stored anywhere.

OF, SF, ZF, PF, CF, (CF) are set according to result.

cmp ax, bx ; ax > bx cf = 0

; ax = bx cf = 0, zf = 1

; ax < bx cf = 1

INC $\Rightarrow$ op1 = op1 + 1

mov AL, 4

INC AL ; AL = 5

RET

SUB $\Rightarrow$ op1 = op1 - op2

mov al, 5

sub al, 1 ; al = 4

ret

SBB $\Rightarrow$ op1 = op1 - op2 - CF

stc

mov al, 5

sbb al, 3 ; al = 5 - 3 - 1 = 1

ret

EX. mov al, 5

mov bl, 5

cmp al, bl ; al = 5, zf = 1

ret

(so equal)

reduce → azaltmak
available → kullanılabılır

data and address needed. (input, output)

1 byte → AL

2 byte → AX

The I/O address is stored in register DX as a 16-bit

address.

→ It's called variable address because it is stored in DX and then used to address the I/O device

EX

IN AL, DX

IN AX, DX

IN AL, DX

IN AX, DX

DX → (port number), between 0-255

for other port numbers, use DX register.

OUT DX, AL

OUT DX, AX

OUT DX, AL

OUT DX, AX

Isolated (direct) and Memory Mapped I/O

↓

- the IN, INS, OUT, OUTS transfer data between the microprocessor's accumulator or memory and the I/O device
- the most common I/O transfer technique
- addresses (ports) are separate from memory

↳ the user can expand the memory to its full size without using any of memory space for I/O devices

- disadvantage: the data transferred between I/O and MP must be accessed by the IN, INS, OUT, OUTS

HANDSHAKING

MP (faster) - I/O device (slower)

Synchronizes the I/O device with the microprocessor.

The software polls or tests the BUSY pin to decide

ex (printer) → if printer is busy, the processor waits

if not, the next ASCII character goes to the printer

polling

- Any instruction that references memory can accomplish the transfer
- The PC does not use it
- Does not use IN, INS, OUT, OUTS instructions
- Advantage: Any memory transfer instruction can access the I/O device
- Disadvantage: a portion of memory system is used as the I/O Map.
- ↳ reduces memory available to applications.

mov bx, 0

add bx, 2 → skip to next element

LOOP

Loop is similar to JMP

mov cx, (number)

loop!

control $CX \neq 0$
; $CX \neq 0$ jumps loop!, $CX = CX - 1$
; $CX = 0$, the next sequential instruction executes.

loop loop!

PROCEDURES

A procedure is a group of instructions that usually performs one task.

- Subroutine, method, or function is an important part of any system's architecture.

A procedure is a reusable section of the software stored in memory once, used as often as necessary.

SYNTAX is the procedure's name

name PROC

; here goes the code

; of procedure

RET

(name) ENDP

Calling a Procedure

CALL instruction is used to call a procedure.

EX ORG 100h

CALL M1

mov ax, 2

RET ; return to operating system

M1 PROC

mov bx, 5

RET ; return to caller

M1 ENDP

end

Call

Transfers the flow of the program to the procedure

Call instruction differs from the JMP instruction because a call saves a return address on the stack.

The call instruction is a combination of a PUSH and a JMP instruction.

When call executes, it pushes the return address on the stack, and then jumps to the procedure.